

TRABAJO PRÁCTICO – IMPLEMENTACIÓN DEL JUEGO “SIMÓN DICE”

Microcontrolador LPC1768 – Máquina de Estados Finita (FSM)

1. Consigna de la Actividad

Desarrollar un programa en lenguaje C para el microcontrolador LPC1769 que implemente el juego “Simón Dice”, utilizando una Máquina de Estados Finita (FSM), temporización con SysTick y manejo de pulsadores mediante interrupciones GPIO.

El sistema debe generar una secuencia de luces utilizando 4 LEDs conectados al puerto P2.0–P2.3. El usuario deberá repetir esa secuencia utilizando 4 pulsadores conectados al puerto P0.0–P0.3.

Cada vez que el jugador complete correctamente una ronda, el sistema agregará un nuevo paso a la secuencia. Si el jugador se equivoca, el juego deberá finalizar y volver al estado de espera. Si completa la longitud máxima definida, deberá declararse victoria.

Requisitos generales:

- Utilizar una FSM principal para organizar la lógica del juego.
- Utilizar una submáquina de estados para mostrar la secuencia de LEDs.
- Emplear temporización no bloqueante mediante SysTick.
- Capturar la pulsación de botones mediante interrupciones externas.
- Mantener el código modular y legible mediante funciones auxiliares.

2. Objetivos

- Aplicar el concepto de Máquina de Estados Finita en un sistema embebido.
- Configurar y utilizar GPIO de entrada y salida.
- Implementar temporización basada en SysTick.
- Gestionar eventos asincrónicos mediante interrupciones.
- Organizar el programa en funciones auxiliares y capas de abstracción.

3. Descripción General del Funcionamiento

El sistema arranca en reposo con todos los LEDs apagados. Al presionar cualquier pulsador, comienza una nueva partida.

El microcontrolador genera una secuencia pseudoaleatoria de valores entre 0 y 3, donde cada valor representa un LED. Esa secuencia se muestra al usuario encendiendo y apagando LEDs con tiempos definidos.

Luego de mostrar la secuencia, el sistema espera la entrada del jugador. Cada pulsación se compara contra la secuencia esperada:

- Si el jugador acierta todos los pasos de la ronda, se agrega un nuevo paso y comienza la siguiente ronda.
- Si el jugador falla, el sistema pasa a Game Over.
- Si se alcanza la longitud máxima, el sistema pasa a Victoria.

4. Estados de la Máquina de Estados Principal

La FSM principal está compuesta por los siguientes estados:

- ESTADO_IDLE
- ESTADO_INICIO
- ESTADO_GENERAR_PASO
- ESTADO_MOSTRAR_SECUENCIA
- ESTADO_ESPERAR_JUGADOR
- ESTADO_VALIDAR_JUGADA
- ESTADO_RONDA_SUPERADA
- ESTADO_GAME_OVER
- ESTADO_VICTORIA

5. Definición de cada Estado

5.1 ESTADO_IDLE

Tarea:

- Mantener todos los LEDs apagados.
- Deshabilitar la lectura del usuario.
- Esperar la señal de inicio de juego.

Condición de transición:

- Si evento_start == 1 → pasar a ESTADO_INICIO.

5.2 ESTADO_INICIO

Tarea:

- Reiniciar variables del juego:
 - longitud_secuencia
 - indice_mostrar
 - indice_jugador
 - boton_presionado
 - evento_boton
 - nivel
 - error_jugada
- Reiniciar referencias temporales y subestados.
- Apagar todos los LEDs.

Condición de transición:

- Transición automática → ESTADO_GENERAR_PASO.

5.3 ESTADO_GENERAR_PASO

Tarea:

- Generar un nuevo valor pseudoaleatorio entre 0 y 3.
- Agregarlo al arreglo secuencia[].
- Incrementar longitud_secuencia.
- Reiniciar el índice de muestra.

Condiciones de transición:

- Si longitud_secuencia < MAX_SECUENCIA → ESTADO_MOSTRAR_SECUENCIA.
- Si longitud_secuencia >= MAX_SECUENCIA → ESTADO_VICTORIA.

5.4 ESTADO_MOSTRAR_SECUENCIA

Tarea:

- Mostrar visualmente la secuencia generada.
- Utilizar una submáquina de estados para encender y apagar LEDs con temporización.

Condición de transición:

- Cuando indice_mostrar >= longitud_secuencia → ESTADO_ESPERAR_JUGADOR.

5.5 ESTADO_ESPERAR_JUGADOR

Tarea:

- Habilitar la lectura del usuario.
- Esperar la pulsación de un botón.
- Mostrar el LED correspondiente al botón presionado.
- Esperar a que el usuario suelte el botón antes de continuar.

Condición de transición:

- Si evento_boton == 1 y el botón fue soltado → ESTADO_VALIDAR_JUGADA.

5.6 ESTADO_VALIDAR_JUGADA

Tarea:

- Comparar boton_presionado con secuencia[indice_jugador].

Condiciones de transición:

- Si el botón es correcto y faltan pasos → ESTADO_ESPERAR_JUGADOR.
- Si el botón es correcto y se completó la ronda → ESTADO_RONDA_SUPERADA.
- Si el botón es incorrecto → ESTADO_GAME_OVER.

5.7 ESTADO_RONDA_SUPERADA

Tarea:

- Incrementar el nivel del jugador.
- Preparar la siguiente ronda.

Condiciones de transición:

- Si longitud_secuencia >= MAX_SECUENCIA → ESTADO_VICTORIA.
- En caso contrario → ESTADO_GENERAR_PASO.

5.8 ESTADO_GAME_OVER

Tarea:

- Apagar todos los LEDs.
- Finalizar la partida.

Condición de transición:

- Transición automática → ESTADO_IDLE.

5.9 ESTADO_VICTORIA

Tarea:

- Apagar todos los LEDs.
- Mantener el sistema en espera hasta reinicio manual.

Condición de transición:

- Si evento_start == 1 → ESTADO_INICIO.

6. Tabla de Estados y Transiciones

7. Submáquina de Estados para Mostrar Secuencia

Dentro de ESTADO_MOSTRAR_SECUENCIA se utiliza una submáquina para controlar el parpadeo de LEDs sin bloquear el programa.

Subestados:

- SUB_ENCENDER_LED
 - Enciende el LED correspondiente a la posición actual.
 - Guarda el tiempo de referencia.
- SUB_ESPERAR_LED_ON
 - Espera TIEMPO_LED_ON milisegundos.
- SUB_APAGAR_LED
 - Apaga todos los LEDs.
 - Guarda un nuevo tiempo de referencia.
- SUB_ESPERAR_LED_OFF
 - Espera TIEMPO_LED_OFF milisegundos.
 - Luego avanza al siguiente elemento de la secuencia.

8. Resumen de Funciones Útiles

8.1 Funciones de Configuración

- void config_botones(void)
Configura P0.0–P0.3 como entradas GPIO con interrupción por flanco descendente.
- void leds_init(void)
Configura P2.0–P2.3 como salidas GPIO.

8.2 Funciones de Manejo de LEDs

- void leds_apagar_todos(void)
Apaga todos los LEDs.
- void led_encender(uint8_t indice)
Enciende un LED específico.
- void led_apagar(uint8_t indice)
Apaga un LED específico.
- void led_mostrar(uint8_t indice)
Apaga todos los LEDs y enciende solo el LED indicado.

8.3 Funciones de Temporización

- uint8_t tiempo_cumplido(uint32_t referencia, uint32_t demora_ms)
Verifica si transcurrió un tiempo determinado desde una referencia.
- void SysTick_Handler(void)
Interrupción periódica que incrementa el contador global ticks_ms.

8.4 Funciones de Lectura de Usuario

- uint8_t boton_sigue_presionado(uint8_t boton)
Verifica si el pulsador continúa físicamente presionado.
- void EINT3_IRQHandler(void)
Atiende las interrupciones de los pulsadores y registra el botón presionado.

8.5 Funciones declaradas pero no implementadas en el código mostrado

- void demora_150ms(void)
Puede utilizarse como antirrebote o retardo bloqueante si el docente lo requiere.

9. Variables Importantes del Sistema

- estado_actual: almacena el estado principal del juego.
- secuencia[]: guarda la secuencia generada.
- longitud_secuencia: cantidad actual de pasos válidos.
- indice_mostrar: índice usado al reproducir la secuencia.
- indice_jugador: índice usado al validar la entrada del jugador.
- boton_presionado: último botón detectado.

- evento_boton: bandera de nueva pulsación.
- evento_start: bandera de inicio de partida.
- nivel: ronda actual.
- error_jugada: indica si el jugador cometió un error.
- ticks_ms: contador global de tiempo en milisegundos.
- tiempo_referencia: marca temporal para comparaciones.
- subestado_mostrar: subestado de la secuencia visual.
- habilitar_lectura_usuario: habilita o bloquea la captura de pulsaciones.

10. Criterios de Evaluación

Se evaluará:

- Correcta implementación de la FSM principal.
- Correcta implementación de la submáquina de estados.
- Uso apropiado de interrupciones GPIO.
- Uso correcto de SysTick para temporización.
- Claridad, modularidad y prolijidad del código.
- Comentarios y legibilidad del programa.

11. Entregables Sugeridos

El alumno deberá entregar:

- Código fuente en C.
- Opcional: simulación o video del funcionamiento en hardware.

Estado	Tarea principal	Condición de transición	Siguiente estado
ESTADO_IDLE	Espera de inicio, LEDs apagados	evento_start == 1	ESTADO_INICIO
ESTADO_INICIO	Reinicialización de variables	Automática	ESTADO_GENERAR_PASO
ESTADO_GENERAR_PASO	Agregar nuevo paso a la secuencia	longitud_secuencia < MAX_SECUENCIA	ESTADO_MOSTRAR_SECUENCIA
ESTADO_GENERAR_PASO	Control de fin de juego	longitud_secuencia >= MAX_SECUENCIA	ESTADO_VICTORIA
ESTADO_MOSTRAR_SECUENCIA	Mostrar secuencia al usuario	Fin de reproducción	ESTADO_ESPERAR_JUGADOR
ESTADO_ESPERAR_JUGADOR	Esperar entrada del jugador	Botón detectado y soltado	ESTADO_VALIDAR_JUGADA
ESTADO_VALIDAR_JUGADA	Comparar entrada con secuencia	Acierto parcial	ESTADO_ESPERAR_JUGADOR
ESTADO_VALIDAR_JUGADA	Comparar entrada con secuencia	Acierto total	ESTADO_RONDA_SUPERADA
ESTADO_VALIDAR_JUGADA	Comparar entrada con secuencia	Error	ESTADO_GAME_OVER

ESTADO_RONDA_SU PERADA	Incrementar nivel	longitud_secuencia >= MAX_SECUENCIA	ESTADO_VICTORIA
ESTADO_RONDA_SU PERADA	Incrementar nivel	En otro caso	ESTADO_GENERAR_P ASO
ESTADO_GAME_OVE R	Finalizar partida	Automática	ESTADO_IDLE
ESTADO_VICTORIA	Esperar reinicio	evento_start == 1	ESTADO_INICIO